NARC ENGINE

How the NARC Engine Works

Architecture & Algorithms of a UCI Chess Engine

Madushan Dissanayake and Claude AI

Technical Design Document · Version 1.0

Contents

1. The big picture
2. How the board is represented (bitboards)
3. Generating moves (magic bitboards)
    - Pseudo-legal generation + legality check
    - Verifying correctness with perft
    - A note on move encoding
4. Making and unmaking moves
5. How the engine evaluates a position
    - Material and piece-square tables
    - Tapered evaluation (the game changes as pieces come off)
    - Beyond material: positional understanding
6. The search: how it "thinks ahead"
    - Minimax and alpha-beta
    - Iterative deepening
    - Aspiration windows
    - Principal Variation Search (PVS)
7. Making the search fast (pruning & reductions)
    - Static Exchange Evaluation (SEE)
8. Move ordering: the secret to speed
9. The transposition table (memory)
10. Quiescence search (avoiding the horizon effect)
11. Time management
12. Talking to the GUI (the UCI protocol)
13. Why it's strong — and how to make it stronger
    - Why it already beats grandmasters
    - Where the strength comes from, in one sentence each
    - How to make it even stronger

A guided tour of the architecture, algorithms, and design decisions behind the NARC Engine — a grandmaster-strength UCI chess engine in ~1,400 lines of C++20.

---

1. The big picture

At heart, a chess engine answers one question: **"What is the best move in this position?"** It does this by combining two ingredients:

- **Search** — looking ahead through millions of possible move sequences.
- **Evaluation** — scoring a position with a number (e.g. "+0.7" = White is better by about ⅔ of a pawn).

The engine plays out moves in its head, far into the future, scores the resulting positions, and assumes both sides play their best. The move that leads to the best *guaranteed* outcome is chosen. This is the **minimax** principle, and almost everything else in the engine is machinery to do it *faster* and *deeper*.

The codebase is organized into five files:

File	Responsibility
`types.h`	Bitboard primitives, attack tables, Zobrist keys, move encoding
`position.h`	The board, make/unmake, move generation, SEE, perft
`eval.h`	Static evaluation (scoring a position)
`search.h`	The search tree, transposition table, time control
`main.cpp`	The UCI protocol that connects to Arena and other GUIs

A typical search explores **1–4 million positions per second** and reaches **18–25 moves deep** within a few seconds.

---

2. How the board is represented (bitboards)

A chessboard has 64 squares. The single most important idea in a modern engine is to represent "where are all the white pawns?" as a **64-bit integer** — one bit per square. This is a **bitboard**.

```
A bitboard for white pawns at the start (bit set = pawn present):

8  . . . . . . . .
7  . . . . . . . .
6  . . . . . . . .
5  . . . . . . . .
4  . . . . . . . .
3  . . . . . . . .
2  X X X X X X X X     ← all eight bits on rank 2 are set
1  . . . . . . . .
   a b c d e f g h
```

The engine keeps a handful of these:

```
u64 byColor[2];   // all white pieces, all black pieces
u64 byPiece[6];   // all pawns, all knights, ... all kings (both colors)
u8  board[64];    // a quick "what piece is on this square?" lookup
```

Why bitboards? Because the CPU can manipulate all 64 squares **in a single instruction**. Want every square a knight on e4 attacks? That's one table lookup. Want to know if two bitboards overlap? One `AND` operation. Counting pieces is one `popcount` instruction. Operations that would be loops over a 2-D array become single, branch-free machine instructions.

Helpers in `types.h` wrap the CPU intrinsics:

```
popcount(b)   // how many bits are set (how many pieces)
lsb(b)        // index of the lowest set bit (find a piece)
poplsb(b)     // pop and return the lowest bit (iterate pieces)
```

---

3. Generating moves (magic bitboards)

For **knights, kings and pawns**, the squares they attack depend only on where they stand, so the engine precomputes a lookup table once at startup: `knightAtt[e4]` instantly gives every square a knight on e4 hits.

Sliding pieces (bishops, rooks, queens) are harder: a rook's reach depends on which pieces *block* it. The classic high-performance solution is **magic bitboards**, and the engine implements them in full (`initMagics` in `types.h`).

The trick works like this:

1. Take the squares whose occupancy *could* block the rook (the "relevant mask").
2. Multiply that occupancy by a carefully chosen "magic" 64-bit constant.
3. Shift the result down to form a small index.
4. Use that index into a precomputed table of attack sets.

The magic numbers are *searched for* at startup — the engine tries random sparse constants until it finds one that maps every possible blocker configuration to a unique, collision-free slot. After that, getting a rook's attacks is just:

```
u64 rookAttacks(int sq, u64 occ) {
    const Magic& m = rookMagics[sq];
    return m.attacks[m.index(occ)];   // multiply, shift, look up — done
}
```

This is what lets the engine generate and verify **~25 million positions per second** (measured via `perft`, below).

Pseudo-legal generation + legality check

`genMoves()` produces all *pseudo-legal* moves (legal except they might leave your own king in check). Rather than expensively proving legality up front, the engine simply plays each move and asks "is my king now attacked?" — if so, the move is taken back and discarded. This make/test/unmake approach is simple and, with fast bitboards, very quick.

A template parameter lets the same code generate either **all moves** (for normal search) or **only captures and promotions** (for quiescence search):

```
genMoves<false>(pos, ml);  // everything
genMoves<true>(pos, ml);   // captures/promotions only
```

Verifying correctness with perft

How do you know move generation has zero bugs? **Perft** (performance test) counts all leaf nodes at a fixed depth and compares against known-correct values published by the chess programming community. The NARC Engine matches **all five** standard test positions exactly — for example, from the start position to depth 6:

```
perft 6: 119,060,324 nodes   ✓ (matches the known value exactly)
```

An exact match across these adversarial positions (which stress en passant pins, promotions, castling through check, etc.) is strong evidence the rules are implemented perfectly.

A note on move encoding

Every move is squeezed into a single **16-bit integer**: 6 bits for the from-square, 6 for the to-square, 2 for the promotion piece, and 2 for a flag (normal / promotion / en passant / castling). Small moves mean more fit in cache, and copying them is nearly free.

---

4. Making and unmaking moves

Search explores millions of positions, so applying and reverting a move must be extremely cheap. The engine never copies the whole board. Instead, `make()` incrementally updates the bitboards and a small `Undo` record captures just enough to reverse it (the captured piece, castling rights, en-passant square, the hash key, and the fifty-move counter). `unmake()` restores them.

Crucially, the **Zobrist hash key** is updated incrementally on the fly. This key is a 64-bit fingerprint of the position, built by XOR-ing in random numbers for each piece-on-square, the side to move, castling rights, and en-passant file. Because XOR is its own inverse, moving a piece is just two XORs (remove from old square, add to new square). This near-unique fingerprint is what makes the transposition table (Section 9) and repetition detection possible.

---

5. How the engine evaluates a position

When the search reaches the bottom of its lookahead, it needs to score the position as a single number. This is the **static evaluation** (`evaluate()` in `eval.h`). The score is always returned **from the side-to-move's perspective** (positive = good for the player about to move).

Material and piece-square tables

The foundation is **material** (a queen ≈ 900 centipawns, rook ≈ 500, etc.) plus **piece-square tables (PSQT)** — a bonus or penalty for *where* each piece sits. A knight in the center is worth more than one in the corner; a pawn on the 7th rank is worth far more than one on the 2nd. The engine uses the well-known **PeSTO** tables, which are tuned values widely used in the community.

Tapered evaluation (the game changes as pieces come off)

A king belongs in the corner during the middlegame (safety) but in the center during the endgame (activity). To handle this, **every term has two values** — a middlegame score and an endgame score — and the engine blends them based on how much material remains (the "phase"):

```
final = (middlegame × phase + endgame × (24 − phase)) / 24
```

With a full board, `phase` is 24 and the middlegame values dominate; as pieces are traded the score smoothly **tapers** toward the endgame values. This single idea dramatically improves endgame play.

Beyond material: positional understanding

On top of material and PSQT, the engine adds the judgments a strong human makes:

- **Mobility** — pieces with more legal squares get a bonus (an active bishop beats a hemmed-in one).
- **Passed pawns** — pawns with no enemy pawns ahead to stop them, scored higher the closer they are to promotion.
- **Isolated & doubled pawns** — structural weaknesses get penalties.
- **Bishop pair** — having both bishops is worth a bonus.
- **Rooks on open files** — rooks love files with no pawns.
- **Tempo** — a small bonus for having the move.

All of these use the same mg/eg tapered pair, so their importance shifts correctly between phases.

---

6. The search: how it "thinks ahead"

This is the engine's brain. It builds a tree of possible futures and uses **alpha-beta pruning** to avoid examining moves that can't matter.

Minimax and alpha-beta

Pure minimax would examine every move sequence — astronomically expensive. **Alpha-beta** keeps two bounds while searching:

- **alpha** — the best score the side to move is already guaranteed elsewhere.
- **beta** — the best the opponent will allow.

If a move proves so good that the opponent would never permit this line (the score reaches **beta**), the engine stops searching the remaining moves here — they cannot change the outcome. This **cutoff** is the single most important speedup in chess search, and with good move ordering it lets the engine search roughly the *square root* of the positions naive minimax would.

The engine uses the **negamax** formulation: a position's value to one side is the negation of its value to the other, so a single routine handles both colors (`-search( ... )` flips perspective on each ply).

Iterative deepening

Rather than searching directly to depth 20, the engine searches to depth 1, then 2, then 3, and so on (`iterativeDeepening()`). This sounds wasteful but isn't: each shallow search is cheap and **fills the transposition table and move-ordering tables**, so the next, deeper search runs dramatically faster. It also means the engine *always has a best move ready* if time runs out mid-search.

Aspiration windows

After a few iterations the engine has a good guess of the score. For the next depth it searches with a **narrow window** around that guess (`±25` centipawns) instead of the full `(−∞, +∞)`. A narrower window produces more cutoffs and a faster search. If the true score falls outside the guess, the window is widened and the search retried.

Principal Variation Search (PVS)

The engine assumes its first (best-ordered) move is the best, and searches it with a full window. Every *other* move is first tested with a **zero-width window** — a cheap test that only asks "is this move worse than what we already have?" Usually the answer is yes and the test is fast. Only on the rare occasion a move looks *better* does the engine re-search it properly.

---

7. Making the search fast (pruning & reductions)

Modern strength comes from aggressively **not** searching unpromising moves. The NARC Engine layers many such heuristics (all in `search()` in `search.h`):

- **Null-move pruning** — "If I let my opponent move twice in a row and I'm *still* winning, this position is so good I can stop here." The engine plays a null (pass) move at reduced depth; if the result still beats beta, it prunes. (Disabled in endgames where zugzwang makes passing tricky.)
- **Reverse futility pruning** — if the static eval is far *above* beta at shallow depth, the position is almost certainly a cutoff; return early.
- **Futility / late-move pruning** — at shallow depth, skip quiet moves that are very unlikely to raise alpha, and skip "late" quiet moves once enough have already been tried.
- **SEE pruning** — skip captures that lose material according to **Static Exchange Evaluation** (see below).
- **Late Move Reductions (LMR)** — moves tried late in the list (after good move ordering) are probably bad, so they're searched at **reduced depth** first. If one surprises us by beating alpha, it's re-searched at full depth. The reduction amount comes from a `log(depth) × log(moveNumber)` formula — reduce more the deeper you are and the later the move.
- **Check extensions** — when a move gives check, the search goes *one ply deeper*, because forcing lines deserve closer inspection.
- **Internal iterative reduction** — if there's no hash move to guide ordering, reduce depth slightly rather than search a poorly-ordered node deeply.

Static Exchange Evaluation (SEE)

Before committing to a capture, the engine asks: *"If we trade off everything on this square, do I come out ahead?"* SEE simulates the full sequence of captures and recaptures (always recapturing with the cheapest attacker) and returns the net material outcome. It's used both to order captures and to prune losing ones.

> ***Together, these techniques mean the engine searches a small, sharply-focused slice of the tree — spending its time on the moves that actually matter and reaching far greater depth than brute force ever could.***

---

8. Move ordering: the secret to speed

Alpha-beta's efficiency depends *entirely* on trying good moves first — a cutoff on the first move searches one move instead of thirty. So before searching, every move is scored and the best is tried first (`scoreMoves` / `pickMove`). The priority order:

1. **The transposition-table move** — the best move found here in a previous (shallower) search. Almost always still best.
2. **Good captures** — ordered by MVV-LVA (capture the most valuable victim with the least valuable attacker) and filtered by SEE.
3. **Killer moves** — quiet moves that caused a cutoff in a *sibling* node at the same depth; they often work again.

4. **History heuristic** — quiet moves are scored by how often they've caused cutoffs across the whole search, with a "gravity" update that lets the statistics adapt over time.
5. **Bad (losing) captures** — searched last.

This ordering is why the engine, despite examining millions of nodes, gets a cutoff on the very first move in the large majority of nodes.

---

9. The transposition table (memory)

The same position often arises via different move orders (a *transposition*). Re-searching it would be pure waste. The **transposition table (TT)** is a large hash table (default 128 MB) that remembers, for positions already searched:

- the best move found,
- the score and how deep it was searched,
- whether that score was exact, a lower bound, or an upper bound.

Each position is indexed by its Zobrist key. On revisiting a position, if the stored search was at least as deep as needed, the engine can **return the saved score instantly** — or at minimum use the stored move to improve ordering.

Because scores near checkmate encode "mate in N plies *from here*," they're adjusted by the current ply when stored and restored (`scoreToTT` / `scoreFromTT`) so mate distances stay correct across transpositions.

---

10. Quiescence search (avoiding the horizon effect)

Imagine the search stops exactly after the engine captures a pawn with its queen — it would happily score that as "+1 pawn," blind to the fact that the queen gets recaptured next move. This is the **horizon effect**.

The fix is **quiescence search** (`qsearch`). When the main search reaches depth zero, instead of evaluating immediately, it keeps searching — but **only captures, promotions, and check evasions** — until the position is "quiet" (no profitable captures remain). Only then does it call the static evaluation. This ensures the engine never misjudges a position in the middle of a trade.

Quiescence has its own speedups: **delta pruning** (skip captures that can't possibly raise alpha even in the best case) and **SEE pruning** (skip captures that lose material). A "stand-pat" score lets the side to move decline to capture if the position is already good enough.

---

11. Time management

In a real game the engine has a clock, so it must decide how long to think. From the GUI's `go wtime ... btime ... winc ...` command (`goCommand` in `main.cpp`), it estimates how many moves remain and budgets accordingly, computing two limits:

- a **soft limit** — after finishing the current depth, if this much time has passed, stop and move (don't start a new, deeper iteration that may not finish);
- a **hard limit** — an absolute cap; the search is interrupted immediately even mid-iteration if reached.

A configurable **Move Overhead** (default 30 ms) is subtracted to absorb GUI and network lag so the engine never flags (loses on time). The search checks the clock every few thousand nodes rather than every node, keeping the check overhead negligible.

---

12. Talking to the GUI (the UCI protocol)

Chess GUIs like **Arena**, Cute Chess, and ChessBase don't care how an engine works internally — they speak the **Universal Chess Interface (UCI)**, a simple text protocol over standard input/output. The engine's `uciLoop()` reads commands line by line:

GUI sends	Engine does
`uci`	Identifies itself (`id name NARC Engine 1.0`) and lists options
`isready`	Replies `readyok` once initialized
`setoption name Hash value 256`	Resizes the transposition table
`position startpos moves e2e4 e7e5`	Sets up the board
`go wtime 60000 btime 60000`	Starts searching; streams `info` lines
`stop`	Halts the search and returns the best move

While searching, the engine streams progress the GUI displays:

```
info depth 19 seldepth 33 score cp 24 nodes 3550472 nps 1325297 time 2679 pv e2e4 e7e5 ...
```

— current depth, score in centipawns (or `mate N`), nodes searched, speed, and the **principal variation** (the line it expects). When done, it sends `bestmove e2e4` . The search runs on a **separate thread** so the engine can still respond to `stop` instantly while thinking.

There are also a few non-UCI console helpers for debugging: `d` prints the board, `eval` prints the static evaluation, and `go perft N` runs the move-generation count test.

---

13. Why it's strong — and how to make it stronger

Why it already beats grandmasters

A grandmaster calculates a few candidate lines maybe 5–15 plies deep, guided by deep intuition, but is fallible and tires. The NARC Engine calculates **every** relevant line **18–25 plies deep**, never miscalculates a

tactic, never blunders from fatigue, and evaluates positions with tuned positional knowledge. In testing it scored **6–4 against Stockfish limited to 2600 Elo** — well above the ~2500 grandmaster threshold — and solved classic combinations (including a queen-sacrifice mate) in milliseconds. Against a human GM it is a heavy favorite.

The strength comes from the *combination*: correct, fast move generation → deep, sharply-pruned search → sound tapered evaluation → never-tiring execution.

Where the strength comes from, in one sentence each

- **Magic bitboards** make move generation fast enough to search millions of positions per second.
- **Alpha-beta + great move ordering** turns an impossible tree into a tractable one.
- **The pruning suite (null-move, LMR, futility, SEE)** focuses that search on the moves that matter, multiplying effective depth.
- **The transposition table** stops the engine from ever solving the same problem twice.
- **Tapered PeSTO evaluation** gives sound positional judgment that adapts from opening to endgame.

How to make it even stronger

The engine is deliberately a clean classical design. The highest-impact next steps would be:

1. **NNUE evaluation** (a small neural network instead of hand-crafted terms): typically **+300–500 Elo**, the single biggest upgrade available.
2. **Multi-threaded search** (Lazy SMP) to use all CPU cores.
3. **An opening book and endgame tablebases** for perfect play at the edges of the game.
4. **Singular extensions** and further search-heuristic tuning.
5. **Texel tuning** of the evaluation weights against a large game database.

This document describes the engine as implemented in `src/`. For build and usage instructions, see README.md.